Roo said

I'll add Playwright with TypeScript support, including Page Object Model and Cucumber integration.

Roo wants to read this file

`src/scripts/prompts.js (lines 420…`

API Request

Roo wants to read this file

`src/scripts/prompts.js (lines 467…`

7 file(s) changed in this conversation +1009 -155

Approve Deny

Type a message...

(@ to add context, / for commands, hold shift to drag in files/images)

Code default Auto-approve off

feature/base* 2 0 Select Postgres Server

Task Completed

I've added comprehensive Playwright with TypeScript support! Now you have complete coverage across all major automation frameworks.

New Additions: Playwright TypeScript Options

1. Playwright TypeScript - Page Object Only (`PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY`)

Generates a single TypeScript Page Object class using Playwright's modern API.

Example Output:

Example Output:

```typescript
import { Page, Locator } from '@playwright/test';

/**
 * Page Object for Login Page
 */
export class LoginPage {
  readonly page: Page;

  // Locators
  private readonly usernameField: Locator;
  private readonly passwordField: Locator;
  private readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameField = page.locator('#username');
    this.passwordField = page.locator('#password');
    this.loginButton = page.locator('#loginBtn');
  }

  async enterUsername(username: string): Promise<void> {
    await this.usernameField.fill(username);
  }

  async enterPassword(password: string): Promise<void> {
    await this.passwordField.fill(password);
  }

  async clickLogin(): Promise<void> {
    await this.loginButton.click();
  }
}
```

Key Features:

- ✅ Modern Playwright locators (`page.locator()`)
- ✅ Async/await patterns throughout
- ✅ TypeScript type safety with `Locator` and `Page` types
- ✅ Readonly page property

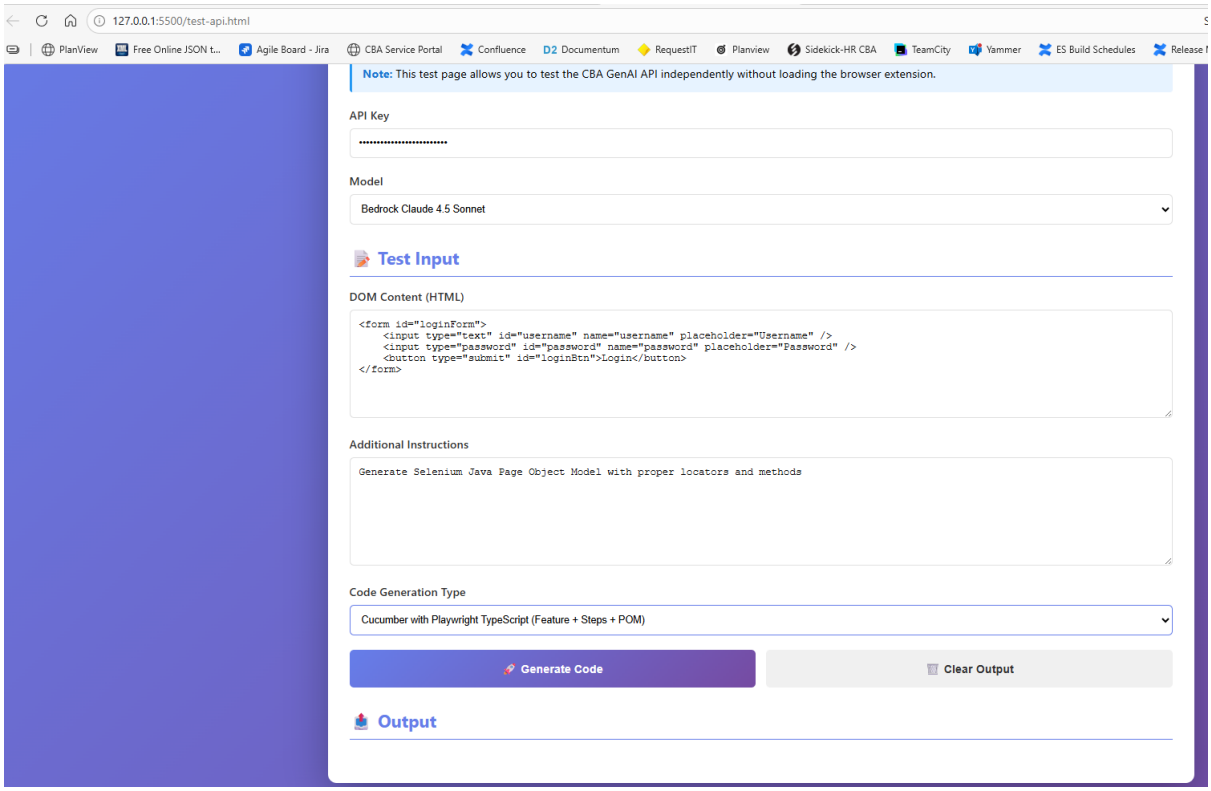
7 file(s) changed in this conversation +1252 -155

Complete Framework Coverage

You now have **7 comprehensive code generation options:**

#	Option	Framework	Language	Files	Use Case
1	**Selenium Java - Page Object Only**	Selenium	Java	1	Java POM library
2	**Selenium C# - Page Object Only**	Selenium	C#	1	C# POM library
3	**Playwright TypeScript - Page Object Only**	Playwright	TypeScript	1	TS POM library ✨ NEW
4	**Cucumber - Feature File Only**	BDD	Gherkin	1	Specifications
5	**Cucumber with Selenium Java Steps**	Selenium + BDD	Java	2	Java BDD tests
6	**SpecFlow with Selenium C# (Complete)**	Selenium + BDD	C#	3	C# BDD tests
7	**Cucumber with Playwright TypeScript (Complete)**	Playwright + BDD	TypeScript	3	TS BDD tests ✨ NEW

Cannot test via extension as its blocked in my organization, tested via test html by passing the DOM manually for test purposes.



Cucumber BDD Test Automation with Playwright & TypeScript

1. Feature File

```gherkin
Feature: Login to Application
  As a registered user
  I want to log into the application
  So that I can access my account

  Background:
    Given I navigate to the login page

  Scenario Outline: Successful login with valid credentials
    When I enter "<username>" in the username field
    And I enter "<password>" in the password field
    And I click the login button
    Then I should be logged in successfully

    Examples:
      | username      | password    |
      | john.smith    | Pass@123    |
      | jane.doe      | Test@456    |
      | admin.user    | Admin@789   |
      | test.account  | Secure@2024 |
```

```
  Scenario Outline: Failed login with invalid credentials
    When I enter "<username>" in the username field
    And I enter "<password>" in the password field
    And I click the login button
    Then I should see an error message

    Examples:
      | username      | password    |
      | invalid.user   | WrongPass1  |
      | john.smith     | incorrect   |
      |              | Pass@123    |
      | jane.doe       |           |

  Scenario: Login form validation
    When I click the login button
    Then I should see validation errors for required fields
```

2. Page Object Model (TypeScript)

```typescript
import { Page, Locator } from '@playwright/test';

/**
 * Page Object Model for Login Page
 * Encapsulates all interactions with the login form
 */
export class LoginPage {
  readonly page: Page;

  // Locators
  private readonly usernameField: Locator;
  private readonly passwordField: Locator;
  private readonly loginButton: Locator;
  private readonly loginForm: Locator;
  private readonly successMessage: Locator;
  private readonly errorMessage: Locator;
  private readonly validationErrors: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameField = page.locator('#username');
    this.passwordField = page.locator('#password');
    this.loginButton = page.locator('#loginBtn');
    this.loginForm = page.locator('#loginForm');
    this.successMessage = page.locator('.success-message, [data-testid="success"]');
    this.errorMessage = page.locator('.error-message, [data-testid="error"]');
```

```
  this.validationErrors = page.locator('.validation-error, .field-error');
}

/**
 * Navigates to the login page
 * @param url - The URL of the login page
 */
async goto(url: string): Promise<void> {
  await this.page.goto(url, { waitUntil: 'domcontentloaded' });
  await this.loginForm.waitFor({ state: 'visible' });
}

/**
 * Enters username into the username field
 * @param username - The username to enter
 */
async enterUsername(username: string): Promise<void> {
  await this.usernameField.waitFor({ state: 'visible' });
  await this.usernameField.clear();
  await this.usernameField.fill(username);
}

/**
 * Enters password into the password field
 * @param password - The password to enter
 */
async enterPassword(password: string): Promise<void> {
  await this.passwordField.waitFor({ state: 'visible' });
  await this.passwordField.clear();
  await this.passwordField.fill(password);
}

/**
 * Clicks the login button
 */
async clickLogin(): Promise<void> {
  await this.loginButton.waitFor({ state: 'visible' });
  await this.loginButton.click();
}

/**
 * Performs complete login action
 * @param username - The username
 * @param password - The password
 */
async login(username: string, password: string): Promise<void> {
  await this.enterUsername(username);
```

```
  await this.enterPassword(password);
  await this.clickLogin();
}

/**
 * Verifies successful login
 * @returns True if login was successful
 */
async isLoginSuccessful(): Promise<boolean> {
  try {
    await this.successMessage.waitFor({ state: 'visible', timeout: 5000 });
    return await this.successMessage.isVisible();
  } catch {
    return false;
  }
}

/**
 * Verifies error message is displayed
 * @returns True if error message is visible
 */
async isErrorMessageDisplayed(): Promise<boolean> {
  try {
    await this.errorMessage.waitFor({ state: 'visible', timeout: 5000 });
    return await this.errorMessage.isVisible();
  } catch {
    return false;
  }
}

/**
 * Gets the error message text
 * @returns The error message text
 */
async getErrorMessage(): Promise<string> {
  await this.errorMessage.waitFor({ state: 'visible' });
  return await this.errorMessage.textContent() || '';
}

/**
 * Verifies validation errors are displayed
 * @returns True if validation errors are visible
 */
async hasValidationErrors(): Promise<boolean> {
  const count = await this.validationErrors.count();
  return count > 0;
}
```

```
  /**
   * Gets the count of validation errors
   * @returns Number of validation errors
   */
  async getValidationErrorCount(): Promise<number> {
    return await this.validationErrors.count();
  }

  /**
   * Checks if the login form is displayed
   * @returns True if form is visible
   */
  async isLoginFormDisplayed(): Promise<boolean> {
    return await this.loginForm.isVisible();
  }

  /**
   * Gets the current page title
   * @returns The page title
   */
  async getPageTitle(): Promise<string> {
    return await this.page.title();
  }

  /**
   * Waits for navigation after login
   */
  async waitForNavigation(): Promise<void> {
    await this.page.waitForLoadState('networkidle');
  }
}
```

3. Step Definitions (TypeScript)

```typescript
import { Given, When, Then, Before, After, setDefaultTimeout } from
'@cucumber/cucumber';
import { chromium, Browser, Page, BrowserContext } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { expect } from '@playwright/test';

// Set default timeout for all steps
setDefaultTimeout(60000);

// Global variables for browser instances
```

```
let browser: Browser;
let context: BrowserContext;
let page: Page;
let loginPage: LoginPage;

// Configuration
const BASE_URL = 'https://example.com';
const BROWSER_OPTIONS = {
  headless: false,
  slowMo: 50,
  args: ['--start-maximized']
};

const CONTEXT_OPTIONS = {
  viewport: { width: 1920, height: 1080 },
  recordVideo: {
    dir: './test-results/videos/',
    size: { width: 1920, height: 1080 }
  },
  screenshot: 'only-on-failure'
};

/**
 * Before hook - Runs before each scenario
 * Initializes browser, context, page, and page objects
 */
Before(async function () {
  browser = await chromium.launch(BROWSER_OPTIONS);
  context = await browser.newContext(CONTEXT_OPTIONS);
  page = await context.newPage();
  loginPage = new LoginPage(page);

  console.log('Browser session started');
});

/**
 * After hook - Runs after each scenario
 * Closes browser instances and cleans up resources
 */
After(async function (scenario) {
  // Take screenshot on failure
  if (scenario.result?.status === 'FAILED') {
    const screenshot = await page.screenshot({
      path: `./test-results/screenshots/${scenario.pickle.name}-${Date.now()}.png`,
      fullPage: true
    });
    this.attach(screenshot, 'image/png');
```

```
  }

  await page?.close();
  await context?.close();
  await browser?.close();

  console.log('Browser session closed');
});

/**
 * Given step - Navigate to login page
 */
Given('I navigate to the login page', async function () {
  await loginPage.goto(BASE_URL);
  const isFormDisplayed = await loginPage.isLoginFormDisplayed();
  expect(isFormDisplayed).toBeTruthy();
  console.log('Navigated to login page');
});

/**
 * When step - Enter username
 * @param username - The username to enter
 */
When('I enter {string} in the username field', async function (username: string) {
  await loginPage.enterUsername(username);
  console.log(` Entered username: ${username}` );
});

/**
 * When step - Enter password
 * @param password - The password to enter
 */
When('I enter {string} in the password field', async function (password: string) {
  await loginPage.enterPassword(password);
  console.log('Entered password');
});

/**
 * When step - Click login button
 */
When('I click the login button', async function () {
  await loginPage.clickLogin();
  console.log('Clicked login button');
});

/**
 * Then step - Verify successful login
```

```
 */
Then('I should be logged in successfully', async function () {
  await loginPage.waitForNavigation();
  const isSuccessful = await loginPage.isLoginSuccessful();
  expect(isSuccessful).toBeTruthy();
  console.log('Login successful');
});

/**
 * Then step - Verify error message is displayed
 */
Then('I should see an error message', async function () {
  const hasError = await loginPage.isErrorMessageDisplayed();
  expect(hasError).toBeTruthy();

  const errorMessage = await loginPage.getErrorMessage();
  console.log(` Error message displayed: ${errorMessage}` );
  expect(errorMessage).toBeTruthy();
});

/**
 * Then step - Verify validation errors for required fields
 */
Then('I should see validation errors for required fields', async function () {
  const hasValidationErrors = await loginPage.hasValidationErrors();
  expect(hasValidationErrors).toBeTruthy();

  const errorCount = await loginPage.getValidationErrorCount();
  expect(errorCount).toBeGreaterThan(0);
  console.log(` Validation errors displayed: ${errorCount}` );
});
```

4. BONUS: Selenium Java Page Object Model

```java
package com.example.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;
```

```java
/**
 * Page Object Model for Login Page using Selenium Java
 * Implements Page Factory pattern with explicit waits
 */
public class LoginPage {

    private WebDriver driver;
    private WebDriverWait wait;

    // Page URL
    private static final String PAGE_URL = "https://example.com";

    // Locators using @FindBy annotations
    @FindBy(id = "username")
    private WebElement usernameField;

    @FindBy(id = "password")
    private WebElement passwordField;

    @FindBy(id = "loginBtn")
    private WebElement loginButton;

    @FindBy(id = "loginForm")
    private WebElement loginForm;

    @FindBy(css = ".success-message, [data-testid='success']")
    private WebElement successMessage;

    @FindBy(css = ".error-message, [data-testid='error']")
    private WebElement errorMessage;

    @FindBy(css = ".validation-error, .field-error")
    private WebElement validationErrors;

    /**
     * Constructor - Initializes WebDriver and PageFactory
     * @param driver WebDriver instance
     */
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        PageFactory.initElements(driver, this);
    }

    /**
     * Navigates to the login page
```

```
 */
public void navigateToLoginPage() {
   driver.get(PAGE_URL);
   wait.until(ExpectedConditions.visibilityOf(loginForm));
}

/**
 * Enters username into the username field
 * @param username The username to enter
 */
public void enterUsername(String username) {
   wait.until(ExpectedConditions.visibilityOf(usernameField));
   usernameField.clear();
   usernameField.sendKeys(username);
}

/**
 * Enters password into the password field
 * @param password The password to enter
 */
public void enterPassword(String password) {
   wait.until(ExpectedConditions.visibilityOf(passwordField));
   passwordField.clear();
   passwordField.sendKeys(password);
}

/**
 * Clicks the login button
 */
public void clickLoginButton() {
   wait.until(ExpectedConditions.elementToBeClickable(loginButton));
   loginButton.click();
}

/**
 * Performs complete login action
 * @param username The username
 * @param password The password
 */
public void login(String username, String password) {
   enterUsername(username);
   enterPassword(password);
   clickLoginButton();
}

/**
 * Verifies if login was successful
```

```
     * @return true if success message is displayed
     */
    public boolean isLoginSuccessful() {
        try {
            wait.until(ExpectedConditions.visibilityOf(successMessage));
            return successMessage.isDisplayed();
        } catch (Exception e) {
            return false;
        }
    }

    /**
     * Verifies if error message is displayed
     * @return true if error message is visible
     */
    public boolean isErrorMessageDisplayed() {
        try {
            wait.until(ExpectedConditions.visibilityOf(errorMessage));
            return errorMessage.isDisplayed();
        } catch (Exception e) {
            return false;
        }
    }

    /**
     * Gets the error message text
     * @return The error message text
     */
    public String getErrorMessage() {
        wait.until(ExpectedConditions.visibilityOf(errorMessage));
        return errorMessage.getText();
    }

    /**
     * Verifies if validation errors are displayed
     * @return true if validation errors exist
     */
    public boolean hasValidationErrors() {
        try {
            return driver.findElements(By.cssSelector(".validation-error, .field-error")).size() >
0;
        } catch (Exception e) {
            return false;
        }
    }

    /**
```

```
 * Gets the count of validation errors
 * @return Number of validation errors
 */
public int getValidationErrorCount() {
   return driver.findElements(By.cssSelector(".validation-error, .field-error")).size();
}

/**
 * Checks if the login form is displayed
 * @return true if form is visible
 */
public boolean isLoginFormDisplayed() {
   try {
      return loginForm.isDisplayed();
   } catch (Exception e) {
      return false;
   }
}

/**
 * Gets the current page title
 * @return The page title
 */
public String getPageTitle() {
   return driver.getTitle();
}

/**
 * Waits for page to load completely
 */
public void waitForPageLoad() {
   wait.until(webDriver ->
      ((org.openqa.selenium.Javascript
```